MODULE 13 · DOMAIN 1 - ARCHITECTURE BUILT FOR THE CLOUD

# Data Lakes, Lakehouse and Analytics Pipelines

Open tables on S3, governed centrally, queried by the right engine, and shared without giving the data away.

DOMAIN 1 of 5 EXAM WEIGHT 25% MODULE 13 of 52

admin@sthithapragna.com

A lakehouse keeps data in open files on S3, and adds table behaviour on top: transactions, schema and governance.

# The lakehouse, layer by layer

01
### Ingest
Kinesis, MSK, AppFlow, DMS land data in S3.

02
### Store
Iceberg tables in S3, or S3 Tables.

03
### Catalogue
Glue Data Catalog holds schemas and locations.

04
### Govern
Lake Formation grants table, column and row access.

05
### Query
Athena, Redshift and EMR read the same tables.

06
### Share
Cross-account grants, or Clean Rooms.

ON THE EXAM

One copy of the data, many engines. The catalogue and the permissions are what make that safe.

# Iceberg tables or plain Parquet files?

PLAIN PARQUET FOLDERS

## Files with a naming scheme

- Append-friendly, rewrite to change
- Deleting one customer rewrites partitions
- Concurrent writers can collide
- Schema changes are fragile

APACHE ICEBERG TABLES

## Files with table semantics

- ACID transactions on S3
- Row-level UPDATE, DELETE and MERGE
- Schema and partition evolution
- Time travel to earlier snapshots

THE TIEBREAKER

Row-level deletes, upserts or concurrent writers on the lake mean Iceberg.

# S3 Tables or self-managed Iceberg?

SELF-MANAGED ICEBERG

## General purpose bucket

- You schedule compaction and cleanup
- Or enable Glue Data Catalog compaction
- Full control of layout
- Bucket and prefix permissions

AMAZON S3 TABLES

## Table buckets

- Iceberg tables as a bucket type
- Automatic compaction and snapshot cleanup
- Table-level permissions
- Up to 3x faster queries, AWS claims

THE TIEBREAKER

Iceberg without running maintenance jobs: S3 Tables. Existing tables and full control: self-managed.

# Lake Formation permissions

- **Grants on databases, tables and columns,** plus data filters for rows.
- **LF-tags** grant by classification, so new tables inherit access from their tags.
- **Cross-account sharing uses AWS RAM,** by named resource or by LF-tag.
- **Resource links** let Athena and Redshift Spectrum query shared tables.
- Same organisation: available at once. Another organisation: an invitation to accept.

> ON THE EXAM
>
> Hundreds of tables and many consumers: tag-based access control. Named grants do not scale.

# Lake Formation sharing or AWS Clean Rooms?

LAKE FORMATION SHARING

## Give access to data

- Consumer can read the shared rows
- Column and row filters limit what
- Inside or across organisations
- Trust the consumer with the data

AWS CLEAN ROOMS

## Share results, not rows

- Data stays in each member's account
- Analysis rules: aggregation, list, custom
- Differential privacy for aggregates
- Neither side sees the other's raw records

THE TIEBREAKER

If a partner must never see your raw records, sharing access is wrong. Clean Rooms shares only permitted results.

# Choosing the engine

| ENGINE | CHOOSE IT WHEN |
| --- | --- |
| Amazon Athena | Serverless SQL on S3, ad hoc, pay per data scanned |
| Amazon Redshift | Warehouse: high concurrency, complex joins, dashboards |
| Redshift Spectrum | Redshift queries that reach into S3 tables |
| Amazon EMR | Spark and other frameworks; EC2, EKS or Serverless |
| AWS Glue ETL | Serverless Spark jobs, crawlers, the Data Catalog |
| Amazon OpenSearch Service | Search and log analytics |

ON THE EXAM

Engines share the catalogue and the tables. Choose by workload shape, not by where the data lives.

# Athena or Redshift?

CHOOSE ATHENA WHEN

## Occasional, serverless

- Ad hoc and infrequent queries
- Pay per terabyte scanned
- No cluster or capacity
- Data already in S3 tables

CHOOSE REDSHIFT WHEN

## Sustained, concurrent

- Hundreds of users all day
- Complex joins, sub-second dashboards
- Serverless or provisioned capacity
- Spectrum still reads the lake

THE TIEBREAKER

All-day dashboards for hundreds of users outgrow Athena. Occasional exploration does not need Redshift.

# Streaming and SaaS ingestion

| SERVICE | CHOOSE IT WHEN |
| --- | --- |
| Kinesis Data Streams | AWS-native streams, ordered by partition key |
| Amazon MSK | Kafka APIs, existing Kafka clients and connectors |
| MSK Express brokers | Up to 3x throughput per broker, no storage to size |
| Amazon AppFlow | Scheduled or event flows from SaaS apps such as Salesforce |
| AWS DMS | Change data capture from databases into the lake |

ON THE EXAM

Existing Kafka clients mean MSK. SaaS sources without writing integration code mean AppFlow.

# Kinesis Data Streams or Amazon MSK?

KINESIS DATA STREAMS

## AWS-native

- AWS SDK producers and consumers
- On-demand or provisioned shards
- Tight Lambda integration
- Least to learn on AWS

AMAZON MSK

## Kafka-compatible

- Existing Kafka clients unchanged
- Kafka Connect and ecosystem tools
- Standard, Express or Serverless
- Portability beyond AWS

THE TIEBREAKER

Kafka already in use means MSK. Starting fresh on AWS with the least to run means Kinesis.

# A serverless processing pipeline

01
### Stream
Kinesis Data Streams receives events.

02
### Transform
Lambda processes batches from each shard.

03
### Land
Results written to Iceberg tables in S3.

04
### Catalogue
Tables registered in the Glue Data Catalog.

05
### Query
Athena queries on demand; nothing runs when idle.

ON THE EXAM

Skill 1.1.3: no servers, scaling per shard, pay per use. Heavy transforms move to Glue ETL jobs.

# Reading the scenario

| WHEN THE QUESTION SAYS | IT IS TESTING |
|---|---|
| Delete one customer's rows across the lake | Iceberg tables |
| Iceberg without maintenance jobs | Amazon S3 Tables |
| Hundreds of tables, share by classification | Lake Formation LF-tags |
| Partner must not see our raw records | AWS Clean Rooms |
| Hundreds of analysts, dashboards all day | Redshift, not Athena |
| Existing Kafka clients and connectors | Amazon MSK |

ON THE EXAM

Analytics questions turn on change, scale of governance, trust and concurrency.

> Meridian Freight's 200 TB data lake is Parquet in S3, partitioned by date. Privacy requests require deleting individual customers' records, which today means rewriting whole partitions overnight.
>
> Several ingestion jobs write to the same datasets concurrently.

## What should the solutions architect recommend?

A. Keep Parquet and rewrite affected partitions with a nightly Glue job

B. Convert the datasets to Apache Iceberg tables and run row-level DELETE statements

C. Filter deleted customers at read time with S3 Object Lambda

D. Load the lake into Amazon Redshift and delete there

◆ PAUSE AND DECIDE

## What should the solutions architect recommend?

A Nightly partition rewrites

*The expensive process the scenario describes, and still unsafe with concurrent writers.*

B Iceberg with row-level DELETE

Deletes touch only affected files, with transactions for concurrent writers.

C S3 Object Lambda

*Hides data rather than deleting it, and is closed to new customers.*

D Move to Redshift

*Abandons the open lake every other engine reads.*

THE SIGNAL

Row-level deletes plus concurrent writers on S3. Both are what a table format exists to provide.

> Meridian Freight's analytics team wants to adopt Apache Iceberg for new datasets. A trial in a general purpose bucket worked, but query performance fell as small files accumulated.
>
> The team does not want to build or schedule maintenance jobs.

**What should the solutions architect recommend?**

A. Schedule EMR jobs to compact files and expire snapshots

B. Switch back to plain Parquet files

C. Store the tables in an Amazon S3 table bucket

D. Move the datasets to Amazon DynamoDB

◆ PAUSE AND DECIDE

## What should the solutions architect recommend?

A Scheduled EMR maintenance

*Fixes the files, but it is exactly the job the team will not build.*

B Back to Parquet

*Gives up the reasons for adopting Iceberg.*

C S3 table bucket

Iceberg with compaction and snapshot cleanup run automatically by AWS.

D DynamoDB

*A key-value database, not an analytics table store.*

THE SIGNAL

Small-file decay plus no appetite for maintenance jobs is the S3 Tables brief.

> Meridian Freight's central data account holds 600 tables. Forty consumer accounts in the organisation need access according to each table's sensitivity: public, internal or restricted.
>
> New tables are added weekly and must be shared correctly without anyone editing grants.

## What should the solutions architect implement?

A S3 bucket policies granting each account access to table prefixes

B Lake Formation named-resource grants for every table to every account

C Copy each permitted table into each consumer account

D Lake Formation LF-tags on tables, with tag expressions granted to consumer accounts

◆ PAUSE AND DECIDE

## What should the solutions architect implement?

A **Bucket policies**

*Object-level access, no column or row control, and policy size limits at this scale.*

B **Named grants**

*Thousands of grants, and every new table needs new ones.*

C **Copies per account**

*Forty copies to keep in step and govern.*

D **LF-tags**

Tag the table once; access follows from its classification.

THE SIGNAL

Access by classification, and new tables shared without editing grants: tag-based access control.

> Meridian Freight and a retail partner want to know how many customers they share and the shipment volume to those customers by region. Each company has its customer data in its own AWS account.
>
> Neither company may see the other's customer records, and results must never identify an individual.

**Which solution meets these requirements?**

A. An AWS Clean Rooms collaboration with aggregation analysis rules and a minimum aggregation threshold

B. A Lake Formation cross-account share of each company's customer table

C. Exchange hashed email addresses through a shared S3 bucket

D. Redshift data sharing between the two companies' warehouses

◆ PAUSE AND DECIDE

## Which solution meets these requirements?

**A** Clean Rooms with aggregation rules

Joint counts by region come out; raw records never leave either account.

**B** Lake Formation share

*Gives the partner readable access to the records it must not see.*

**C** Hashed emails in S3

*Hashes of common data can be reversed by matching, so records leak.*

**D** Redshift data sharing

*Shares readable tables, which is the same problem as option B.*

THE SIGNAL

Joint answers without exposing either side's records is the purpose of Clean Rooms.

> About 300 Meridian Freight analysts use dashboards over shipment data all working day, with complex joins across eight large tables. The dashboards run on Athena against the lake.
>
> Query charges have tripled and users complain of slow dashboards.

**What should the solutions architect recommend?**

A Create more Athena workgroups to spread the load

B Serve the dashboards from Amazon Redshift, using Spectrum for older history in the lake

C Run Presto on a long-running Amazon EMR cluster

D Index the tables in Amazon OpenSearch Service

◆ PAUSE AND DECIDE

## What should the solutions architect recommend?

A More Athena workgroups

*Organises and limits spend; each query still scans and costs the same.*

B Redshift with Spectrum

Built for concurrent, join-heavy dashboards; history stays in the lake.

C Presto on EMR

*A cluster you tune and operate, to rebuild what a warehouse provides.*

D OpenSearch

*Search and log analytics, not relational joins across tables.*

THE SIGNAL

Sustained concurrency and complex joins are a warehouse workload. Spectrum means the lake is not abandoned.

> Meridian Freight runs a self-managed Apache Kafka cluster on premises, with dozens of producers, consumers and Kafka Connect connectors. Throughput is growing fast and broker storage management takes constant effort.
>
> The applications do not use Kafka Streams, and must not be rewritten.

## Where should the company move its streaming platform?

A Amazon Kinesis Data Streams in on-demand mode

B Amazon MSK with Standard brokers and provisioned storage

C Amazon MSK with Express brokers

D Self-managed Kafka on EC2 with EBS volumes

◆ PAUSE AND DECIDE

## Where should the company move its streaming platform?

A Kinesis Data Streams

*Every Kafka client and connector would have to be rewritten.*

B MSK Standard brokers

*Kafka-compatible, but storage still has to be provisioned and watched.*

C MSK Express brokers

Kafka APIs unchanged, higher throughput per broker, no storage to manage.

D Kafka on EC2

*Moves the same storage burden into AWS.*

THE SIGNAL

Existing Kafka keeps MSK. Growing throughput and storage pain point to Express brokers.

# What to carry out of this module

- **Iceberg** for row-level change and concurrent writers; **S3 Tables** to skip maintenance.
- **Lake Formation LF-tags** for governance at scale; RAM for cross-account.
- **Clean Rooms** when partners must not see each other's raw records.
- **Athena** for occasional queries; **Redshift** for concurrent dashboards.
- **MSK** for existing Kafka; **Kinesis** for AWS-native; **AppFlow** for SaaS.

> ON THE EXAM
>
> This whole module is new to the professional exam. Older SAP courses do not teach it.